

MESSAGING & EVENTS

Doorbreken van synchroniteit t.b.v. kwaliteit

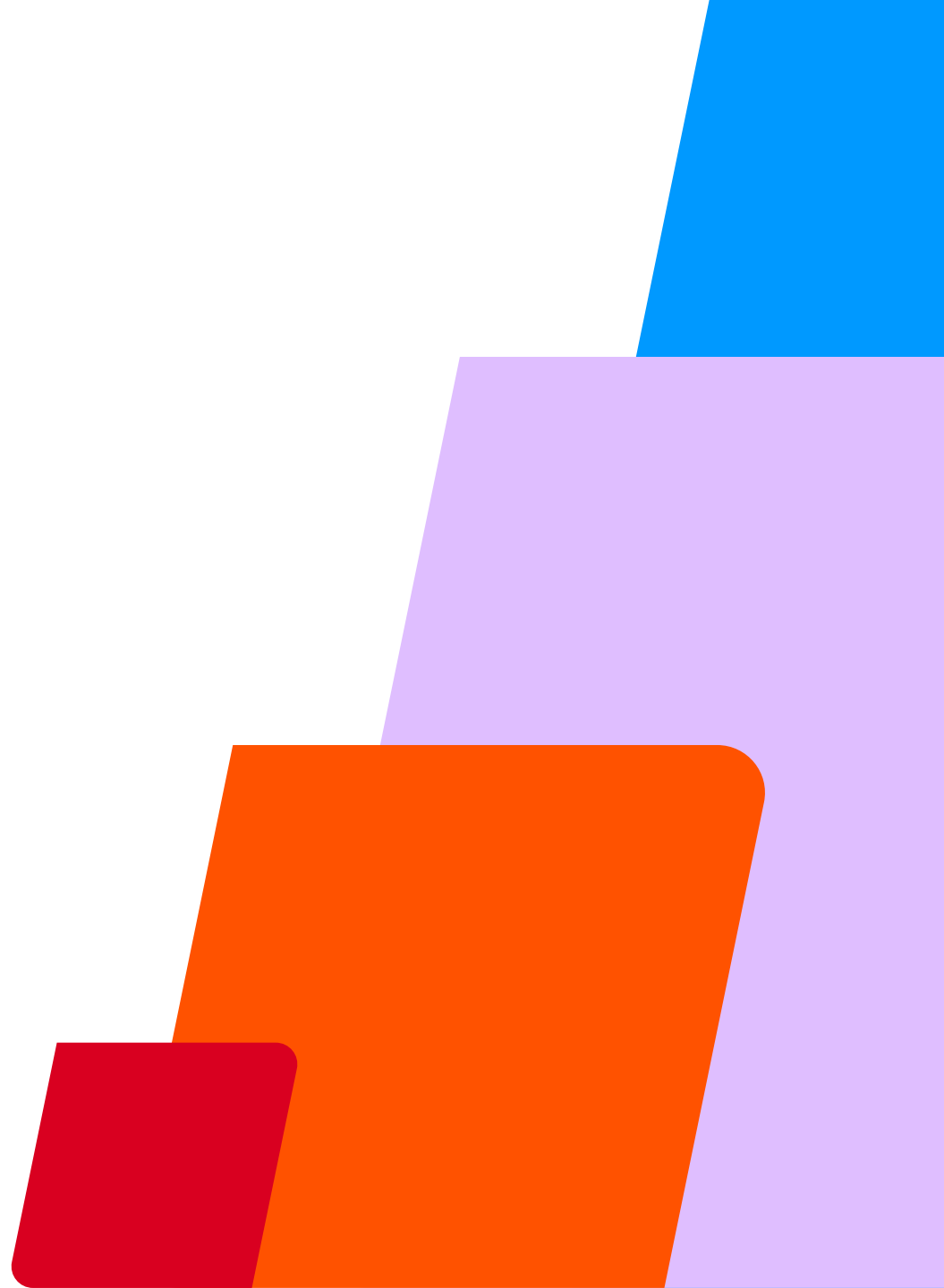


LESDOEL

Je bent in staat de relatie tussen messages & events uit te leggen.

Je kunt een messaging in de basis implementeren door het gebruik van queues en weet dit te koppelen aan architectuurkenmerken of niet-functionele requirements.

Je bent in staat uit te leggen hoe messaging bijdraagt aan de ontkoppeling van systemen en hoe deze kunnen worden ingezet in relatie tot een architectuurstijl.



AGENDA

- Rubric Highlight: Realisatie verantwoording.
- (A)synchrone communicatie tussen systemen
- Message Queues
- Events



DEFEND TRUTH.

REALISATIE VERANTWOORDING

RECENT NIEUWS



The New York Times

Goodbye, \$165,000 Tech Jobs. Student Coders Seek Work at Chipotle.

As companies like Amazon and Microsoft lay off workers and embrace A.I. coding tools, computer science graduates say they're struggling to land tech jobs.



[Home](#) [News](#) [Sport](#) [Business](#) [Technology](#) [Health](#) [Culture](#) [Arts](#) [Travel](#) [Earth](#) [Audio](#) [Video](#) [Live](#)

Computer science graduates struggle to secure their first jobs

22 August 2025

Share

Save

Add as preferred on Google

Joe Fay

Technology Reporter

NOS Nieuws • Vrijdag 24 april, 02:10

Moederbedrijf Instagram, Facebook en WhatsApp ontslaat duizenden medewerkers

Het Amerikaanse techbedrijf Meta gaat wereldwijd zo'n 10 procent van zijn werknemers ontslaan. Daarmee belanden zo'n 8000 medewerkers van het moederbedrijf achter Instagram, Facebook en WhatsApp op straat. Het personeel krijgt over een maand te horen wie wordt ontslagen en wie mag blijven.

Het massaontslag werd als eerste gemeld door persbureau Bloomberg en later bevestigd door de techgigant. Het besluit heeft te maken met de opkomst van kunstmatige intelligentie (AI). Enerzijds gaat Meta vele miljarden extra investeren in nieuwe datacentra. Anderzijds neemt AI taken over van medewerkers.

OH NO WE'RE DOOMED....

Natuurlijk niet...

- De laatste ontwikkelingen volgen was altijd al noodzakelijk in de IT
 - 2000 – Internet Bubble
 - 2008 – Smartphones
 - 2012 – Cloud Computing

... en dan nog vele 'micro'-developments door de jaren heen (talen, architectuurstijlen, etc.)



WAAROM VERTEL JE ONS DIT?

- Het onderwijs kan niet zo snel meebewegen als de buitenwereld (feit).
- Docenten ontbreekt het aan echte hands-on ervaring met de laatste tools en ontwikkelingen (feit)

Niemand weet wat de impact is op de toekomst, maar we kunnen al een beetje terugkijken en de 'trajectory' zien van hoe dit loopt.

Waarom we denken dat er een belangrijke rol voor jou blijft

- De adoptie in de buitenwereld gaat traag (persoonlijke observaties bij stages).
- Er blijven voorlopig voldoende IT-problemen die moeten worden opgelost
- Het ging al nooit om enkel om de code... maar het is wel de wel iets!

WAT VERWACHT JE VAN ONS ?

Voel je vrij (als je je dat nog niet voelde) om AI-tools in te zetten. Je bent niet verplicht dit te doen.

Reflecteer op het (niet) gebruiken ervan:

- Wat betekent het voor je (on)afhankelijkheid?
- Welke tijdsbesparing heb je (niet) gerealiseerd?
- Bestendig de opgedane kennis in een advies voor toekomstige projecten.

Deliverables (GROEP): *Beschrijving van ingezette tooling inclusief bijvoorbeeld System Instructions, Rules, Guidelines en prompt logs.*

CGI (INDIVIDUEEL): *Verwacht vragen zoals hierboven.*

RUBRIC CRITERIA: REALISATIE VERANTWOORDING

(OPEN RUBRIC ON BRIGHTSPACE)

MEER LEZEN/LUISTEREN

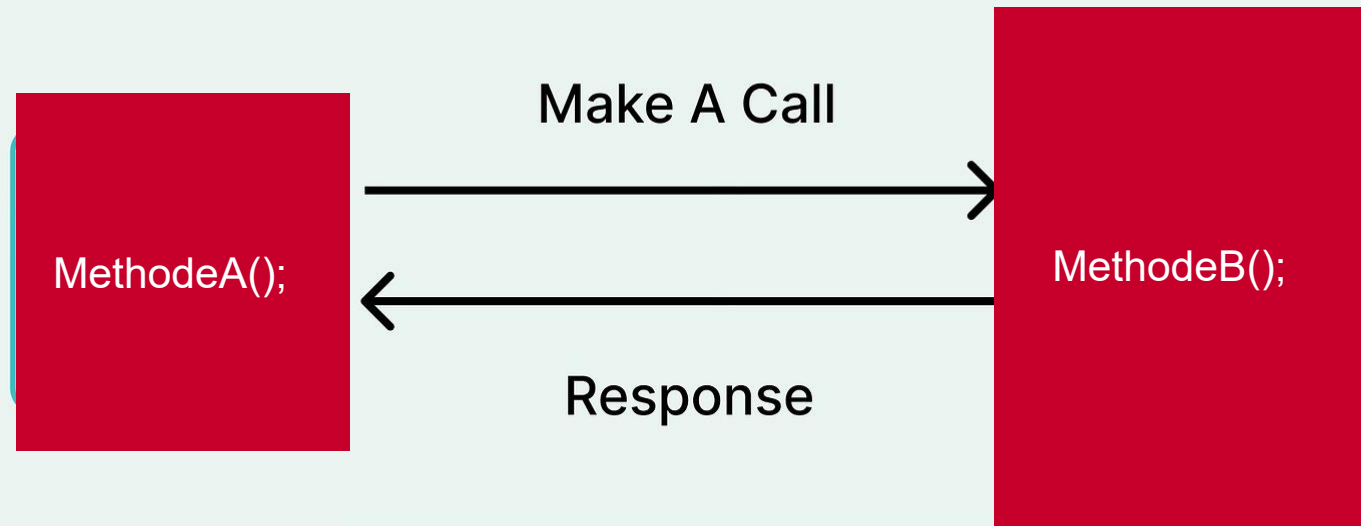
- [2506.08872] Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task (MIT)
- Redefining the Software Engineering Profession for AI – Russinovich (CTO Microsoft Azure), Hanselman (VP Microsoft Developer Community)
- Hoe één baan in 18 maanden totaal veranderde (Podcast AI Report)
- How AI assistance impacts the formation of coding skills (Anthropic)



(A)SYNCHRONE COMMUNICATIE TUSSEN SYSTEMEN

TOT NU TOE ...

Synchronous Communication



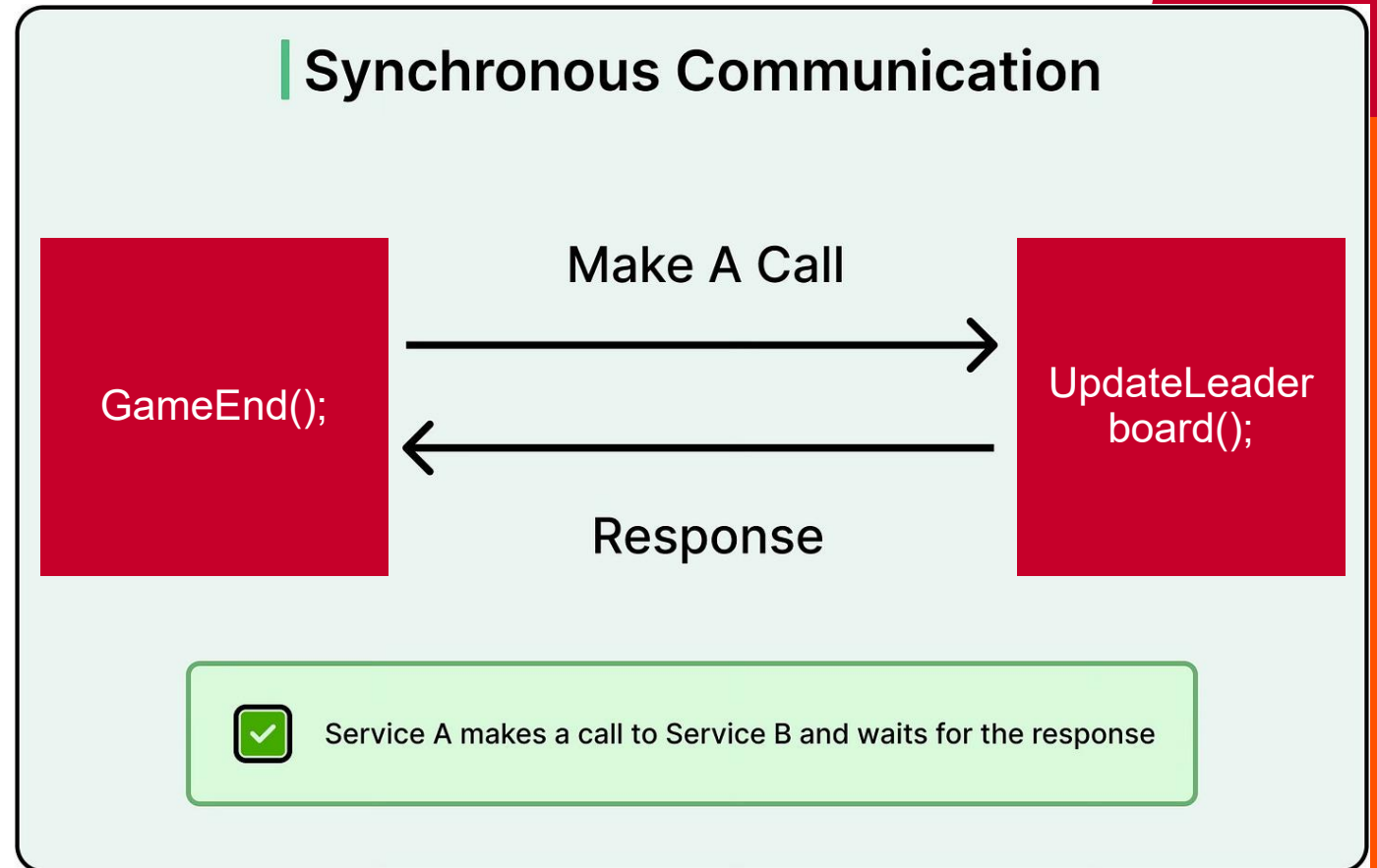
Service A makes a call to Service B and waits for the response

DAAR IS NIETS MIS MEE....

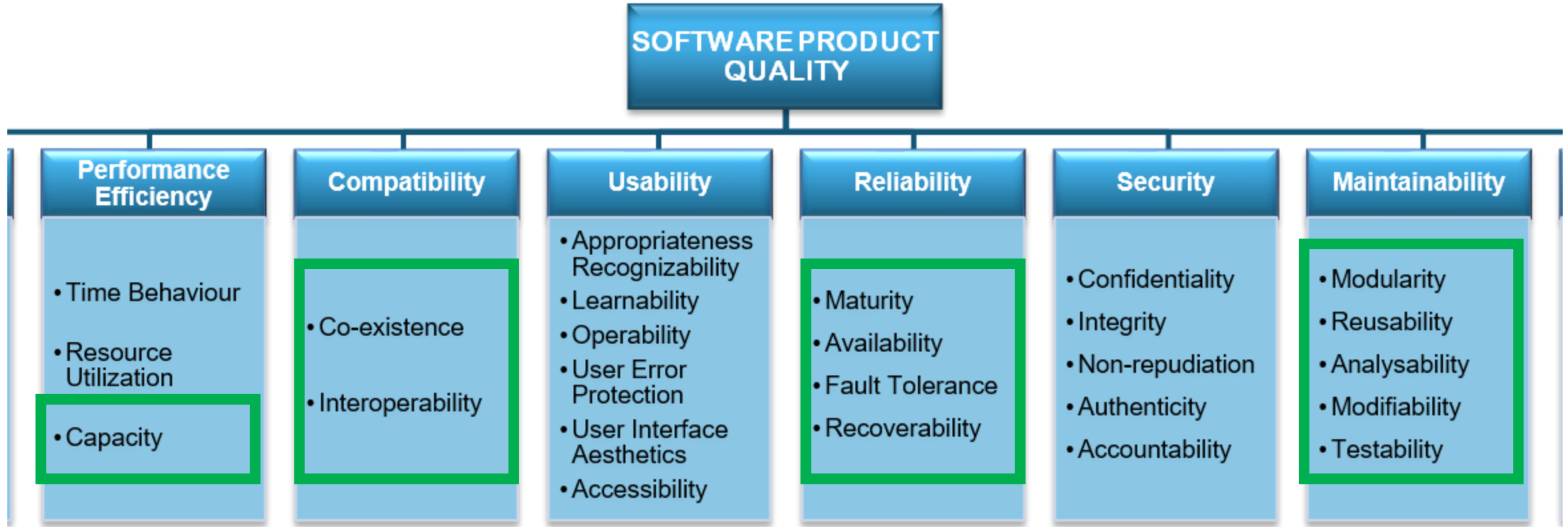
Maar wat als:

- Het bijwerken van de ranglijst 30 seconden duurt?
- Er per minuut 100 wedstrijden worden afgerond (bijvoorbeeld bij *online gaming*)?
- Er uit het bijwerken van een ranglijst bijwerken nieuwe synchrone calls ontstaan (e-mails versturen)?
- Het ranglijst-systeem niet beschikbaar is?

Wat is dan de oplossing?



SYNCHRONITEIT BELEMMERT (SOMS) KWALITEIT



Een asynchrone architectuur *kan* bijdragen aan bovenstaande kwaliteitskenmerken

***WAT BETEKENIT DIT VOOR
DE ARCHITECTUUR?***



MESSAGE QUEUES

WAT IS EEN QUEUE?

QUEUES

Een wachtrij voor berichten (messages) die gevuld worden door een *producer* die *first-in first-out** worden verwerkt door één of meerdere *consumers*.

* *FIFO is standaard, maar andere configuraties zijn mogelijk*

WAT IS EEN MESSAGE?

EEN MESSAGE BESTAAT UIT

Headers — metadata over het bericht: id, timestamp, type, source, correlationId

Routing informatie — exchange, routing key, reply-to adres

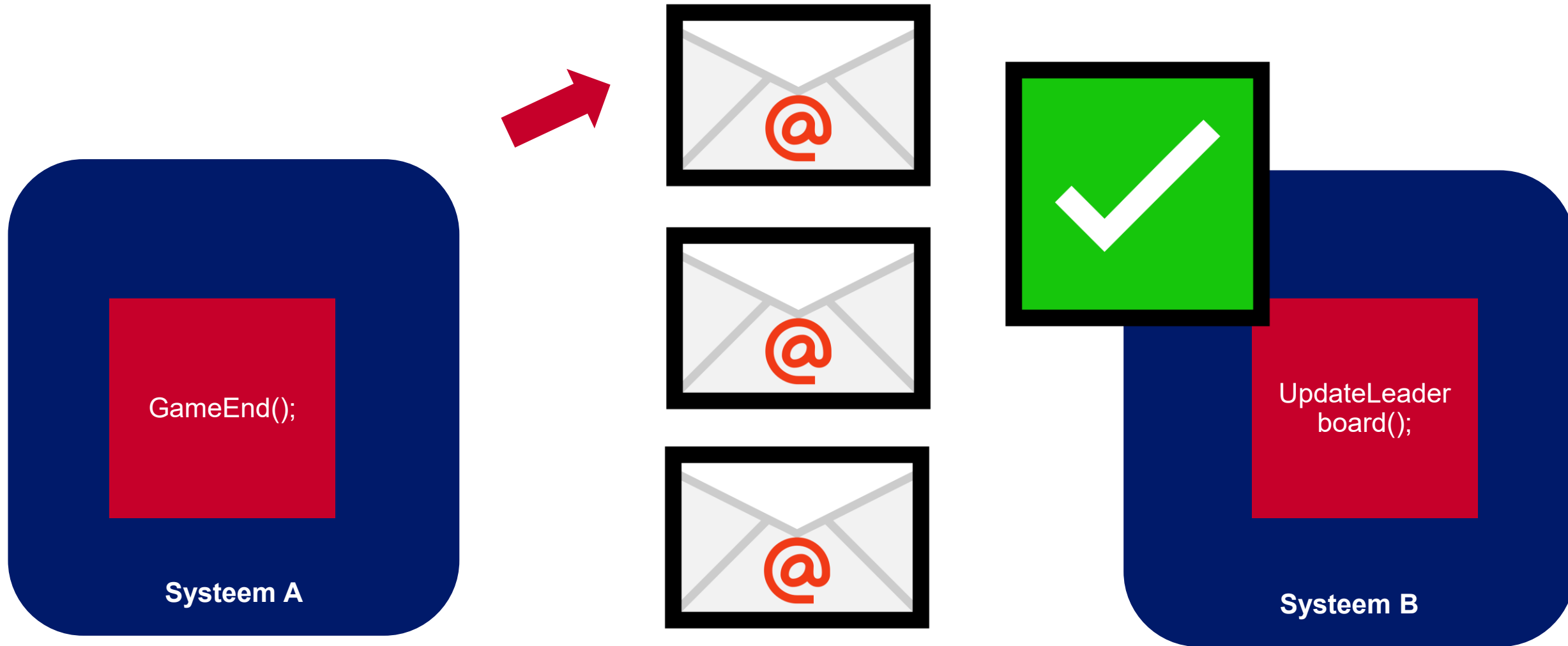
Properties — durable, priority, TTL (time to live), content-type

Payload — de daadwerkelijke inhoud van het bericht, bijvoorbeeld JSON

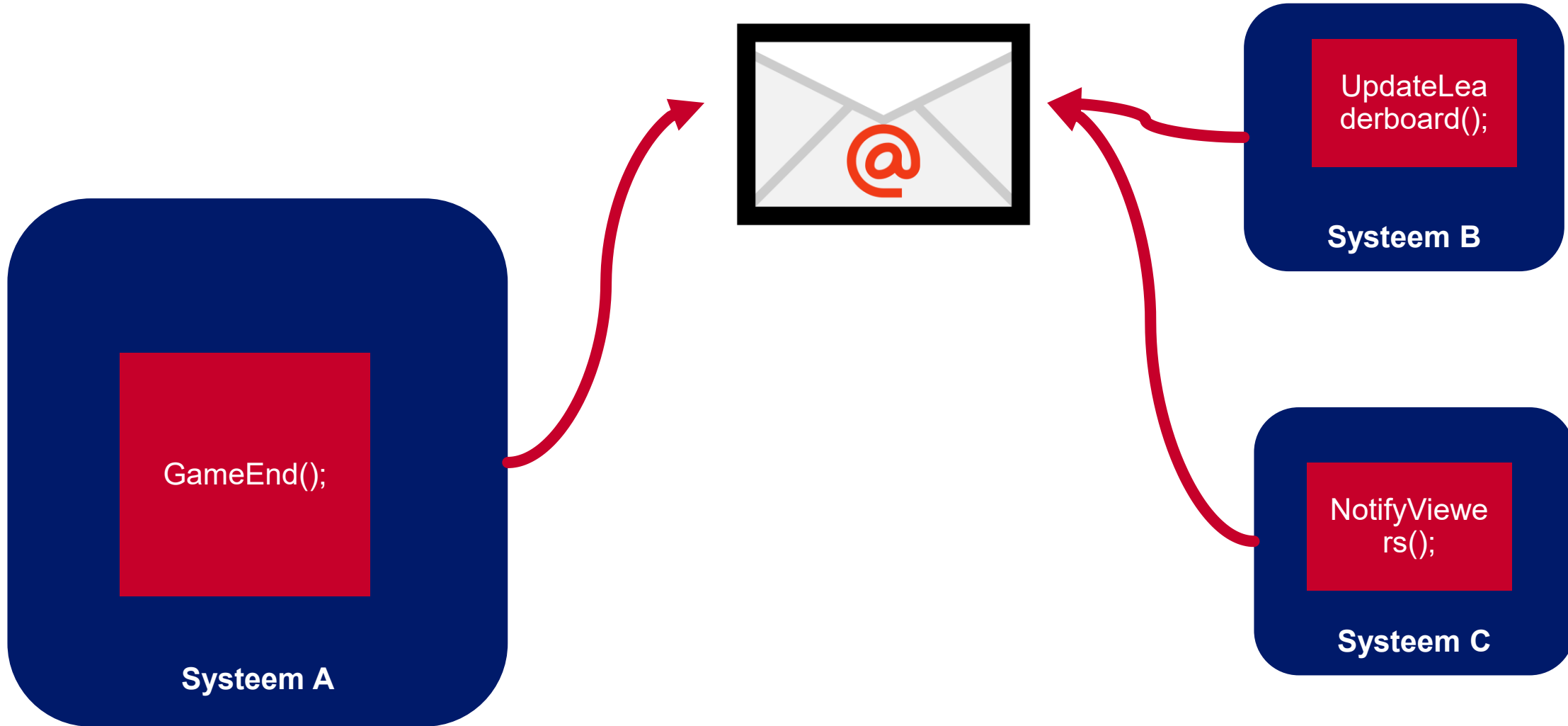
Acknowledgement info — delivery tag waarmee de consumer ack of nack stuurt

```
json
{
  "headers": {
    "eventId": "b7e2d4f1-93bc-4a1e-c7d1-987654321def",
    "eventType": "GameWon",
    "timestamp": "2026-03-28T14:32:00Z",
    "source": "gameserver-eu-west-3",
    "correlationId": "session-9a4c2d1e"
  },
  "routing": {
    "exchange": "game.events",
    "routingKey": "game.won",
    "replyTo": null
  },
  "properties": {
    "durable": true,
    "priority": 1,
    "ttl": 3600000,
    "contentType": "application/json"
  },
  "payload": {
    "gameId": "game-7791",
    "winnerId": 42,
    "winnerUsername": "xX_Destroyer_Xx",
    "durationSeconds": 184,
    "finalScore": 3800
  }
}
```

ONTKOPPELING VAN SYSTEMEN IN TIJD (TEMPORAL COUPLING)



ONTKOPPELING VAN SYSTEMEN IN GEDRAG (BEHAVIORAL COUPLING)



SOFTWARE ENGINEERS HOUDEN REKENING MET..

Topologie 👉 Point 2 Point messaging of publish/subscribe (pub/sub)

Security 👉 Messaging infrastructure betekent duplicatie van data. Hoe beveilig je deze data?

Berichtverwerking

- Idempotency 👉 hetzelfde bericht twee keer verwerken geeft een dubbele actie
- Duplicate delivery 👉 brokers garanderen *at least once*, niet *exactly once*
- Poison messages 👉 een bericht dat altijd faalt en de queue blokkeert
- Message ordering 👉 berichten worden niet altijd verwerkt in de volgorde van verzending

Betrouwbaarheid

- Lost messages 👉 bericht verdwijnt als queue niet durable is of ack ontbreekt
- Retry storms 👉 bij een storing proberen alle consumers tegelijk opnieuw, wat de broker overbelast

Gelijktijdigheid

- Competing consumers 👉 meerdere consumers pakken hetzelfde bericht op
- Stale data 👉 een event bevat data die al verouderd is tegen de tijd dat het verwerkt wordt

RABBITMQ



- **AMQP** 👉 Advanced Message Queuing Protocol (vergelijkbaar met HTTP, maar dan voor Queue producers/consumers)
- **Exchange** 👉 wanneer je gebruik wilt maken van publish/subscribe. Ontkoppelt *producers* en *consumer* in gedrag. Voor point 2 point gebruik je de default exchange.
- **Routing key** 👉 Een label dat de producer op een bericht plakt. De exchange gebruikt dit label om te beslissen naar welke queue het bericht gaat.
- **ACK / NACK** 👉 Bevestig dat een bericht is verwerkt of juist niet?
- **Dead-letter queue** 👉 Als een bericht na X pogingen niet verwerkt kan worden, belandt het in de dead-letter queue. Daar kun je het inspecteren, herverwerken of weggooien 👉 Event as first-class citizen

Let's take a look

in de voorbereiding heb je een fork/clone moeten maken van [rminnaar/dotnet-rabbitmq: A composition of RabbitMQ examples for C# .NET developers.](#)

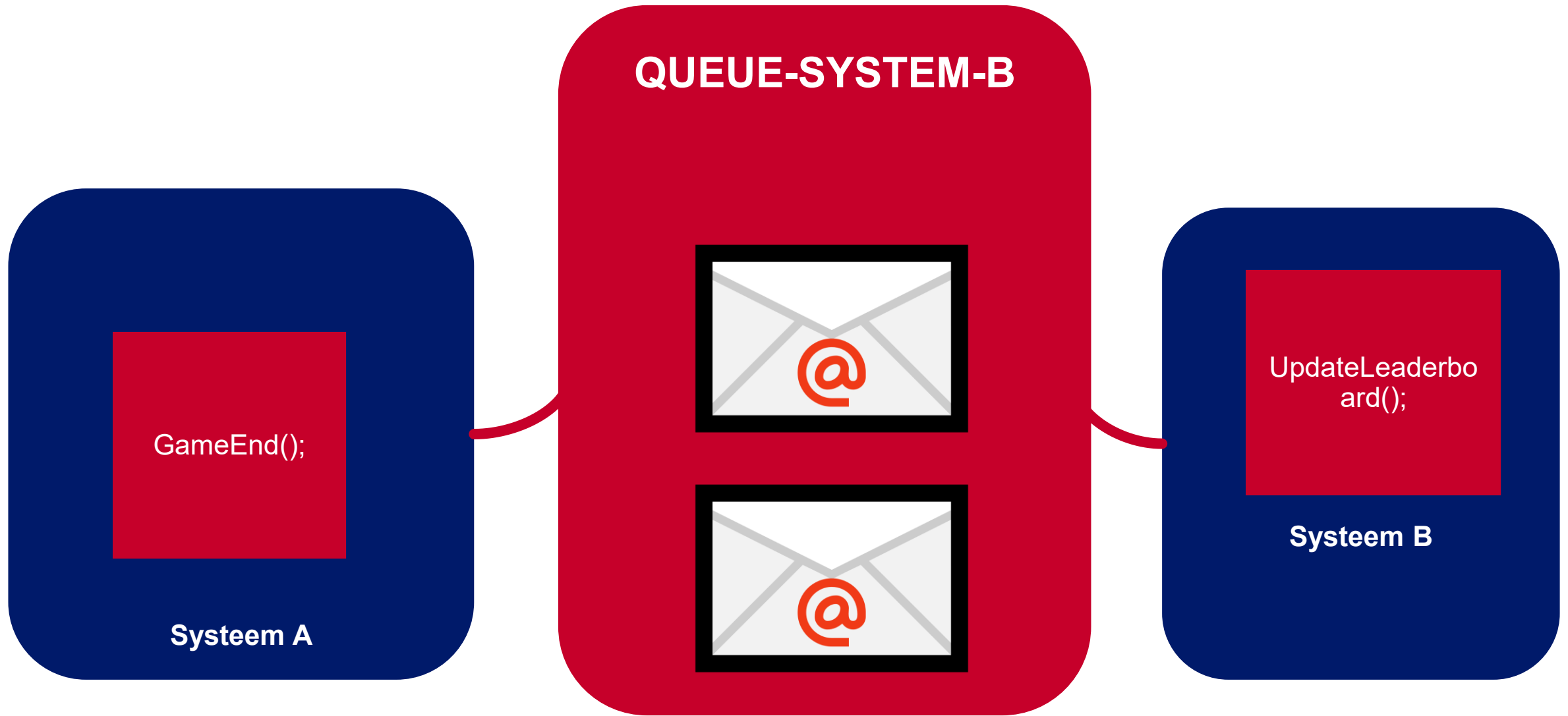
Start met:

`docker compose up`

Debug met:

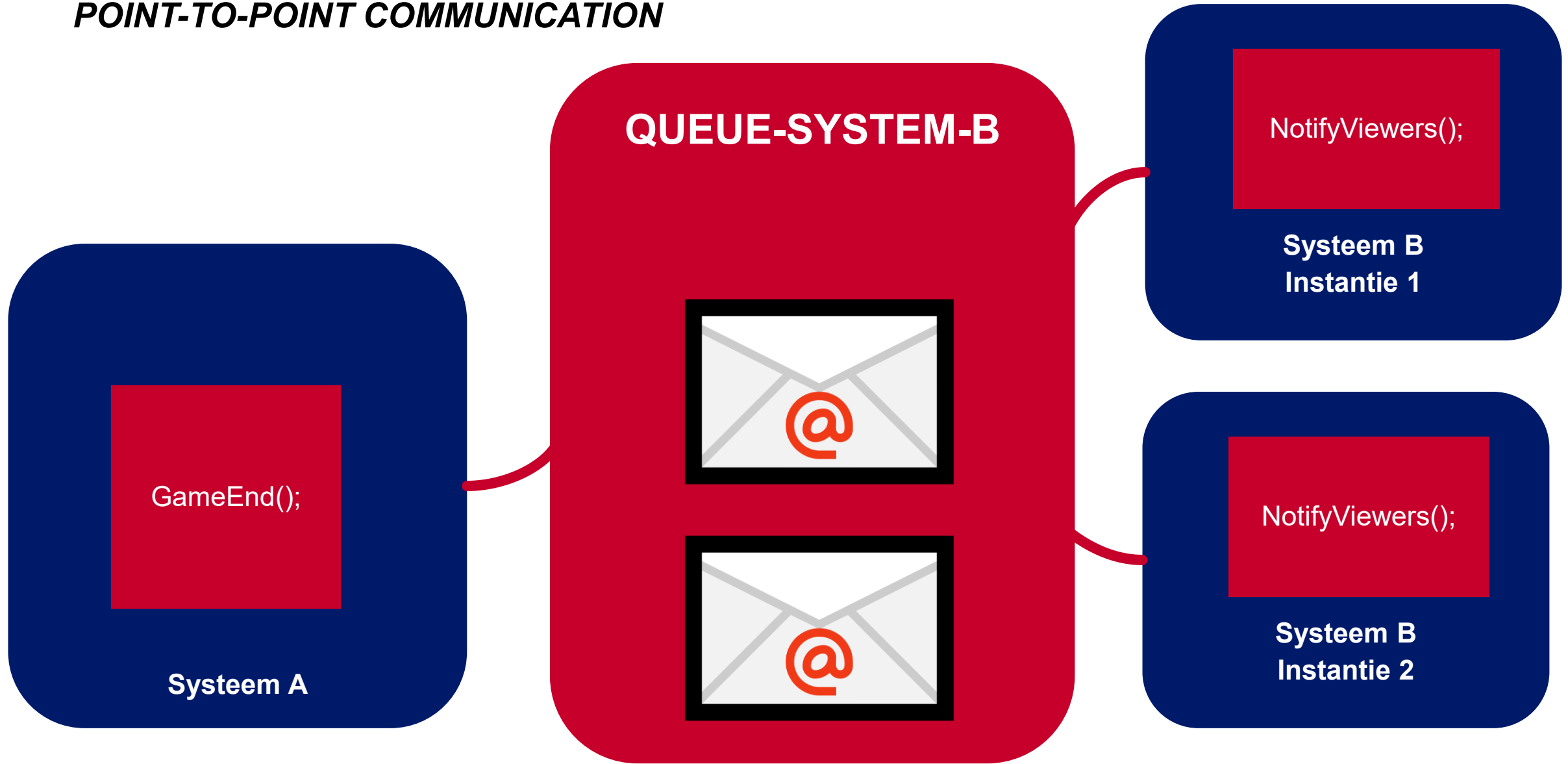
Vanuit Visual Studio 🖱️ rechtermuisknop op project 🖱️ start new instance

POINT-TO-POINT COMMUNICATION



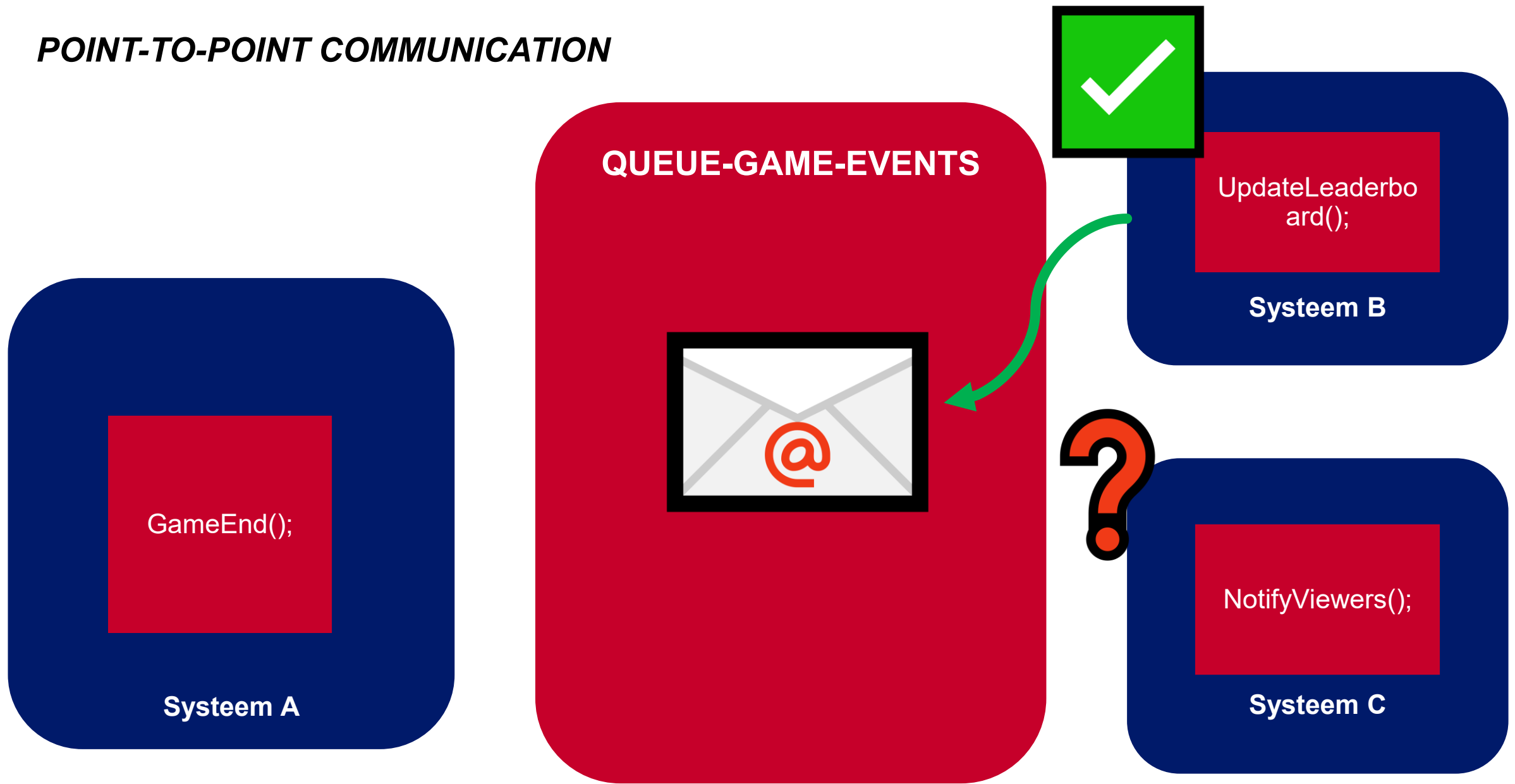
TEMPORAL **×** **BEHAVIORIAL**

POINT-TO-POINT COMMUNICATION



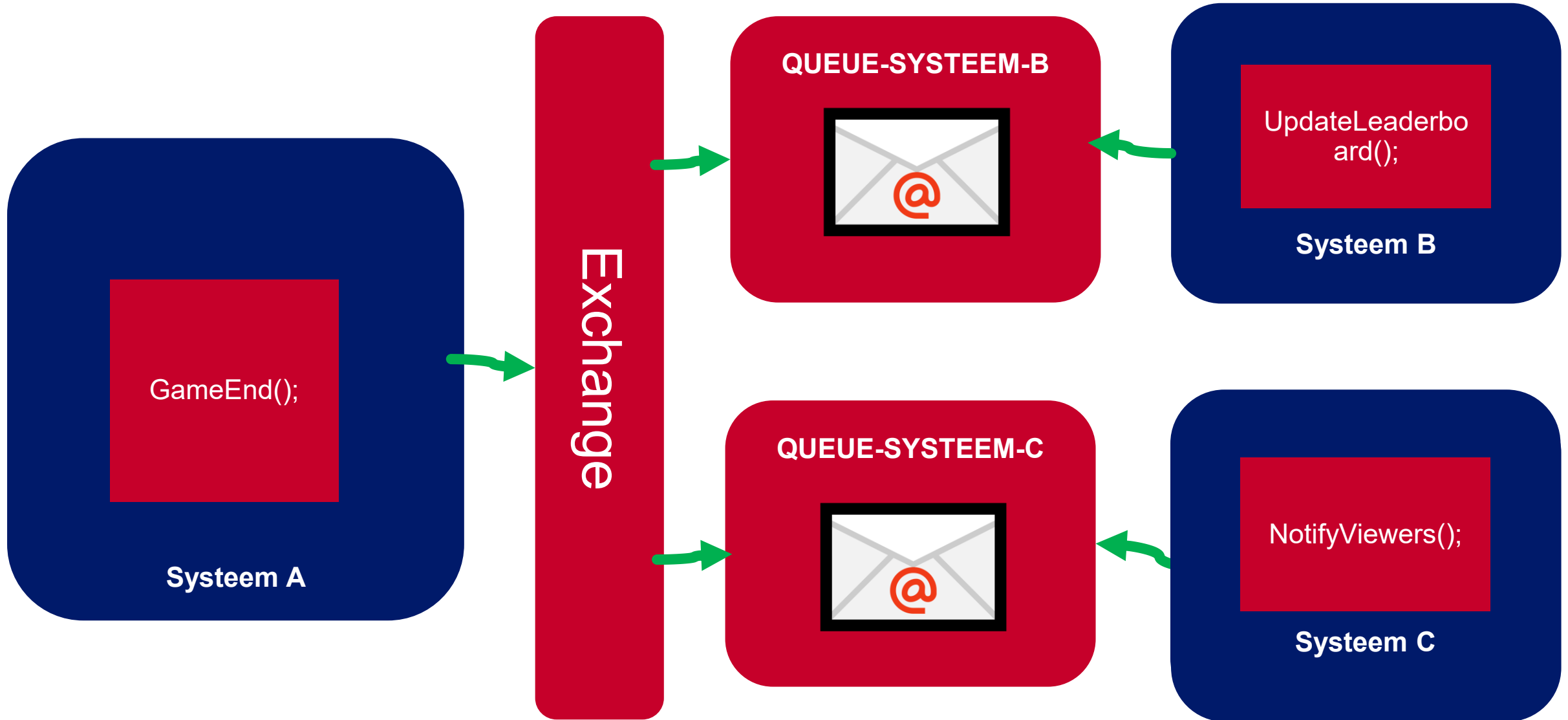
COMPETING CONSUMERS

POINT-TO-POINT COMMUNICATION



✗ COMPETING CONSUMERS

PUBLISH/SUBSCRIBE



TEMPORAL



BEHAVIORIAL

ABSTRACTIES

Queues zijn infrastructuur en vereisen werken met bytes, verbindingen en kanalen.

Er zijn abstracties die het werken met queues eenvoudiger maken. Ze zijn een laag boven op bijvoorbeeld de RabbitMQ of andere implementaties.

.NET 👉 MassTransit

Python 👉 Celery

TypeScript 👉 BullMQ

Zonder MassTransit

RabbitMQ.Client — low-level AMQP

```
var factory = new ConnectionFactory
{
    HostName = "localhost"
};

using var connection =
    factory.CreateConnection();
using var channel =
    connection.CreateModel();

channel.QueueDeclare(
    queue: "notifications",
    durable: true,
    exclusive: false,
    autoDelete: false);

var body = Encoding.UTF8.GetBytes(
    "AppointmentReminder");

var props = channel.CreateBasicProperties();
props.Persistent = true;

channel.BasicPublish(
    exchange: "",
    routingKey: "notifications",
    basicProperties: props,
    body: body);
```

Met MassTransit

Broker-agnostische abstractie

```
// Startup
services.AddMassTransit(x => {
    x.AddConsumer<NotificationConsumer>();
    x.UsingRabbitMq((ctx, cfg) => {
        cfg.ConfigureEndpoints(ctx);
    });
});

// Publisher
await bus.Publish(
    new AppointmentReminder(
        AppointmentId: id));

// Consumer
public class NotificationConsumer
    : IConsumer<AppointmentReminder>
{
    public async Task Consume(
        ConsumeContext<AppointmentReminder> ctx)
    {
        await Send(ctx.Message);
    }
}
```

GROEPSDISCUSSIE

Onderzoek

- Welke mogelijkheden voor abstractie heeft de door jullie gekozen tech stack?
- Hoe komt data vanaf de gekozen integratiemethode binnen op de module?
- Hoe zet je messaging in t.a.v. (niet) functionele requirements.

Uitkomst

- Leg de uitkomst vast in een ADR.





EVENTS

WAT ZIJN EVENTS?

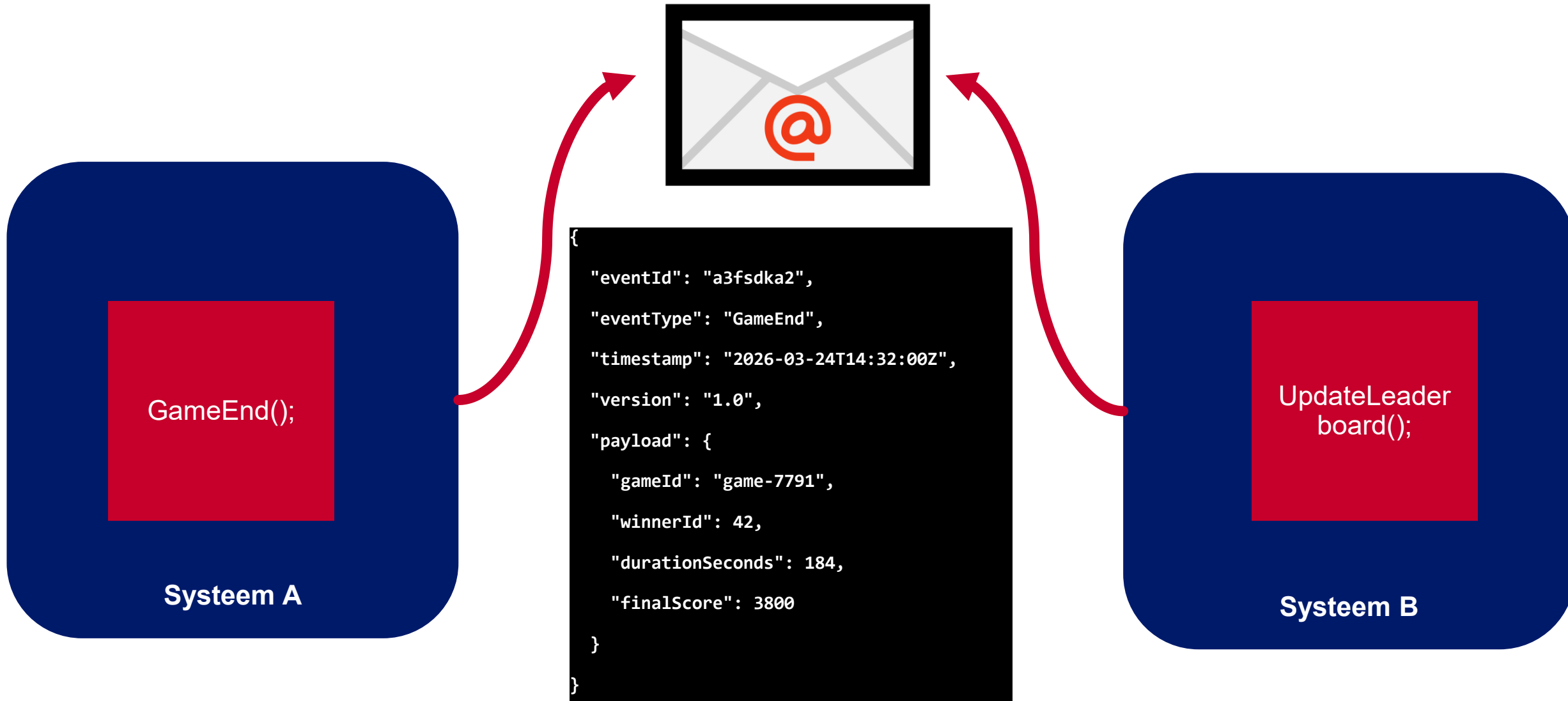
DEFINITIE VAN EEN EVENT

Een event is een onveranderlijk gegeven over iets wat al heeft plaatsgevonden - een feit, geen verzoek.

Drie kenmerken maken het een event

- Verleden tijd – GameEnded, OrderPlaced, AppointmentScheduled
- Onveranderlijk - een event verandert nooit na publicatie.

VOORBEELD VAN EEN EVENT



THIN

VS

FAT

```
{
  "eventId": "a3f9c1d2-84ab-4e2f-b6c0-123456789abc",
  "eventType": "GameWon",
  "timestamp": "2026-03-28T14:32:00Z",
  "version": "1.0",
  "payload": {
    "gameId": "game-7791"
  }
}
```

```
{
  "eventId": "b7e2d4f1-93bc-4a1e-c7d1-987654321def",
  "eventType": "GameWon",
  "timestamp": "2026-03-28T14:32:00Z",
  "version": "1.0",
  "payload": {
    "gameId": "game-7791",
    "winnerId": 42,
    "winnerUsername": "xX_Destroyer_Xx",
    "durationSeconds": 184,
    "finalScore": 3800,
    "opponentId": 17,
    "opponentUsername": "CasualGamer99",
    "mapName": "Dust II",
    "ranked": true
  }
}}
```

THIN

Voordelen

- ✓ Minder risico op *schema drift* vanwege het kleinere schema (Producer / Consumer begrijpen hetzelfde schema).
- ✓ Producer hoeft niet te beslissen wat consumers nodig hebben.
- ✓ Minder kans op *stale data* omdat additionele calls zorgen dat de laatste versie van de data wordt opgehaald.

Nadelen

- ✗ Event zelf bevat doorgaans te weinig data voor consumers om hun 'werk' te kunnen doen. Ofwel additionele synchrone calls nodig, hetgeen schaalbaarheid moeilijker maakt en ook temporale coupling herintroduceert.

VS

FAT

- ✓ Consumer is volledig zelfstandig: geen extra synchrone calls
- ✓ Betere fault tolerance: consumer werkt ook als de producer tijdelijk down is

Nadelen

- ✗ De messaging infrastructuur verwerkt meer data dan nodig is voor de consumers.
- ✗ Data die wordt verwerkt door de consumer kan verouderd zijn ten tijde van verwerking (*stale data*)

WHAT DO WE MEAN BY EVENT-DRIVEN?

- **Systemen die reageren op wat er is gebeurd** - niet op wat er gevraagd wordt)
- **Asynchrone communicatie** - systemen zijn van elkaar losgekoppeld in tijd en gedrag)
- **Events as first class citizens** - events vereisen een expliciete afhandeling en gaan niet zomaar verloren
- **Events** representeren concepten uit de echte wereld.

WHAT DO YOU MEAN BY EVENT-DRIVEN?

De term *event* komt vaker terug in de wereld van software engineering. Iedere groep pakt één van de concepten en legt uit hoe het zich verhoudt of waar het verschilt van *Event-driven architecture*.

- **Eventual Consistency**
- **Event Sourcing**
- **C# Events**
- **Event storming**

VOOR- EN NADELEN EVENT-DRIVEN ARCHITECTURE

Voordelen

- Het biedt de optie om systemen van elkaar te ontkoppelen in tijd (*temporal*) en gedrag (*behaviorial*)
 - *Temporal: Systeem A is niet afhankelijk of Systeem B 'up' is. (fault tolerance, recoverability, co-existence)*
 - *Behaviorial: Systeem A hoeft niet te weten wat Systeem B precies aan data nodig heeft of welke andere systemen er geïnteresseerd zijn (modularity, modifiability)*
- Het kan bijdragen aan een schaalbaarder (performance) systeem die in staat is grotere hoeveelheden acties op een betrouwbare manier te verwerken (*availability, capacity*)
- Event is een *first-class citizen* en vereist expliciete afhandelbaar (gaat niet uit zichzelf verloren) en representeren gebeurtenissen uit de echte wereld.

Nadelen

- Event-driven systemen zijn in zijn geheel lastiger te debuggen.
- Asynchrone communicatie vereist andere keuzes in het ontwerp van het systeem t.a.v. het bedrijfsproces (wanneer is welke informatie beschikbaar?)
- Loskoppeling in tijd (*temporal*) en gedrag (*behaviorial*) vereist backwards compatibility, en dat betekent meer complexiteit.
- Het werken met (gepersisteerde) events werkt het beste met speciaal daarvoor ontworpen infrastructuur (queues) en dat betekent meer complexiteit.
- Volgordelijkheid van events is lastiger te coördineren.

SPRINT 2

Zie Brightspace